BUSINESS AI STACK · CHAPTER 25

Next.js

A practical guide with steps, examples and a checklist. Read the chapter, then build the example.

CHAPTER 25 · BUSINESS AI STACK

Next.js: customer-facing dashboards

Next.js is a React framework for building websites and web apps. Use it when a customer or your team needs a dashboard that logs in, shows live data and works on a phone.

When to use Next.js

- A dashboard that customers log into to see their orders, enquiries or reports.
- A public site that needs fast pages, search engine ranking and server-side data.
- A tool your team uses that reads data from Supabase or an API.

The core ideas, in plain words

- **Pages and routes.** Each file in the `app/` folder becomes a page at the matching address. `app/dashboard/page.js` becomes `/dashboard`.
- **Server and client components.** Components run on the server by default, which keeps secrets off the browser. Add `"use client"` only where you need clicks, forms or live updates.
- **Data fetching.** Read data on the server, then pass it to the page. Never put a secret key in client code.
- **Environment variables.** Store keys in `.env.local` and never commit that file to GitHub.

Build a simple enquiry dashboard

1. Create the project: `npx create-next-app@latest enquiry-dashboard` and accept the defaults.
2. Create a Supabase project and a table `enquiries` with the columns `name`, `phone`, `message`, `status` and `created_at`.
3. Add the Supabase URL and the publishable key to `.env.local`. Use the publishable key only; never use the service-role key in the browser.
4. In `app/dashboard/page.js`, read the latest 50 enquiries on the server and render them as a table with status labels.
5. Protect the page: only show it after sign-in, and make sure the database rules (row level security) allow each user to see only their own rows.

```
// app/dashboard/page.js (server component, simplified)
import { createClient } from "@supabase/supabase-js";
const supabase = createClient(process.env.NEXT_PUBLIC_SUPABASE_URL,
process.env.NEXT_PUBLIC_SUPABASE_KEY);
export default async function Dashboard() {
  const { data } = await supabase.from("enquiries").select("*").order("created_at", { ascending: false
}).limit(50);
  return <table>{data.map(e => <tr key={e.id}><td>{e.name}</td><td>{e.status}</td></tr>)}</table>;
}
```

COMMON MISTAKES

Putting a secret key in a client component. Forgetting row level security, so every user sees every row. Shipping a dashboard with no sign-in.

Checklist

- Pages load without errors on a phone.
- No secret keys in the code or the browser.
- Only signed-in users see the dashboard.
- The project is in GitHub, ready for Chapter 28 (deployment).

Reference: the Next.js documentation (nextjs.org/docs) and the Supabase documentation for the client library.